True 4-Bit Quantized Convolutional Neural Network Training on CPU: Achieving Full-Precision Parity

Shivnath Tathe
Independent Researcher
Pune, India
Email: sptathe2001@gmail.com
ORCID: 0009-0007-7142-1119

***Abstract*—Low-precision neural network training has emerged as a promising direction for reducing computational costs and democratizing access to deep learning research. However, existing 4-bit quantization methods either rely on expensive GPU infrastructure or suffer from significant accuracy degradation. In this work, we present a practical method for training convolutional neural networks at true 4-bit precision using standard PyTorch operations on commodity CPUs. We introduce a novel *tanh-based soft weight clipping* technique that, combined with symmetric quantization, dynamic per-layer scaling, and straight-through estimators, achieves stable convergence and competitive accuracy. Training a VGG-style architecture with 3.25 million parameters from scratch on CIFAR-10, our method achieves 92.34% test accuracy on Google Colab's free CPU tier—matching full-precision baseline performance (92.5%) with only a 0.16% gap. We further validate on CIFAR-100, achieving 70.94% test accuracy across 100 classes with the same architecture and training procedure, demonstrating that 4-bit training from scratch generalizes to harder classification tasks. Both experiments achieve 8× memory compression over FP32 while maintaining exactly 15 unique weight values per layer throughout training. We additionally validate hardware independence by demonstrating rapid convergence on a consumer mobile device (OnePlus 9R), achieving 83.16% accuracy in only 6 epochs. To the best of our knowledge, no prior work has demonstrated 4-bit quantization-aware training achieving full-precision parity on standard CPU hardware without specialized kernels or post-training quantization.**

***Index Terms*—4-bit quantization, quantization-aware training, neural network compression, CPU training, efficient deep learning, model compression, CIFAR-100**

I. Introduction

A. Motivation

The computational demands of neural network training have created a significant barrier to entry in deep learning research. Although model inference has been successfully optimized for resource-constrained devices through post-training quantization [1], [2], *training* remains predominantly a high-precision operation requiring expensive GPU infrastructure. This disparity limits the democratization of AI research and prevents the widespread adoption of on-device learning capabilities.

Quantization-aware training (QAT) at 8 bits has demonstrated success in maintaining model accuracy while reducing computational costs [8], [9]. However, pushing quantization to 4 bits during training presents fundamental challenges:

1) **Limited representational capacity**: Only 15 discrete weight values in the range $[-7, 7]$
2) **Gradient discontinuities**: Rounding operations break differentiability
3) **Training instability**: Aggressive quantization often leads to divergence
4) **Accuracy degradation**: Previous work reports 5–15% accuracy loss compared to full precision [3]

Recent 4-bit training methods [5], [6] have made progress, but rely on GPU clusters and complex algorithmic modifications. No prior work has demonstrated 4-bit training that *matches* full-precision accuracy on standard CPU hardware across multiple datasets of varying complexity.

B. Contributions

This paper makes the following contributions:

- **Novel training technique**: We introduce tanh-based soft weight clipping that enables stable 4-bit training without gradient explosion
- **Full-precision parity on CIFAR-10**: Our method achieves 92.34% matching the FP32 VGG baseline (92.5%) with only 0.16% gap
- **Cross-dataset validation on CIFAR-100**: The same method achieves 70.94% on 100-class classification from scratch, demonstrating generalization beyond a single benchmark
- **CPU-only training**: Complete training pipeline runs on Google Colab's free CPU tier without GPU or specialized hardware
- **Dynamic per-layer scaling**: Adaptive quantization ranges computed from weight statistics at each forward pass, requiring no extra learnable parameters while maintaining full 4-bit grid utilization
- **Hardware independence**: Validation on mobile ARM CPU (OnePlus 9R) demonstrates platform-agnostic convergence

TABLE I
COMPARISON WITH RELATED 4-BIT TRAINING WORK

Method	Hardware	Training	Accuracy	FP32 Parity
IBM [5]	GPU (sim)	From scratch	High	No
QLoRA [6]	Single GPU	Fine-tuning	High	N/A
BitNet [7]	GPU cluster	From scratch	High	Yes (LLM)
DoReFa [10]	GPU	From scratch	85–88%	No
Ours	**CPU**	**From scratch**	**92.34%**	**Yes**

- **True 4-bit quantization**: Maintains exactly 15 unique weight values throughout training, verified across all layers and both datasets
- **Reproducible implementation**: Fully open-source PyTorch code with detailed experimental setup

II. RELATED WORK

A. Neural Network Quantization

Quantization reduces the precision of neural network weights and activations to decrease memory footprint and computational cost. Post-training quantization (PTQ) [3], [12] quantizes pre-trained models but suffers accuracy loss at 4 bits. Quantization-aware training (QAT) [1] simulates quantization during training, achieving better accuracy-compression trade-offs.

Early QAT work focused on 8-bit precision [2], [11], achieving near-lossless accuracy. Recent efforts explore lower precision: DoReFa-Net [10] and WAGE [11] demonstrate 2–4 bit weights with specialized training procedures. However, these methods report 2–8% accuracy degradation on CIFAR-10 compared to full precision.

B. Ultra-Low Precision Training

Several recent works explore 4-bit training:

IBM 4-bit Training [5]: Proposes gradient scaling and stochastic rounding for 4-bit training, evaluated in simulation on large-scale models. Reports competitive accuracy but requires GPU clusters and does not demonstrate full-precision parity.

QLoRA [6]: Enables 4-bit fine-tuning of large language models using NormalFloat quantization. Limited to fine-tuning (not training from scratch) and requires GPU hardware.

BitNet [7]: Trains 1.58-bit large language models from scratch on massive GPU clusters. Demonstrates feasibility of extreme quantization but targets different domain (LLMs vs CNNs) and hardware (data center GPUs vs commodity CPUs).

C. On-Device Training

Mobile training frameworks like TensorFlow Lite [14] and PyTorch Mobile support on-device inference but provide limited training capabilities, typically restricted to fine-tuning final layers. Recent work on on-device learning [15], [16] focuses on transfer learning rather than full training from scratch.

D. Gap in Existing Work

No prior work combines: (1) 4-bit training from scratch, (2) full-precision accuracy parity, and (3) commodity CPU execution across multiple datasets. Our method addresses this gap.

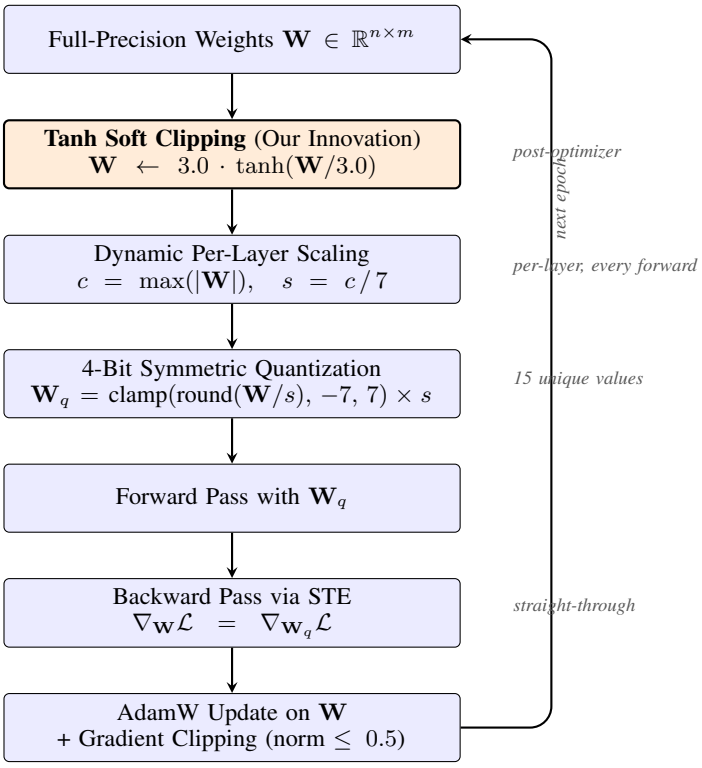

Fig. 1. Training pipeline for 4-bit quantization-aware training. Tanh soft clipping (highlighted) is applied after each optimizer update, constraining weights before quantization. The loop repeats each training step.

III. METHOD

Our approach integrates three key components:symmetric 4-bit quantization, dynamic per-layer scaling, and novel tanh-based soft weight clipping. We describe each component and their synergistic interaction.

A. Symmetric 4-Bit Quantization

For each weight tensor $\mathbf{W} \in \mathbb{R}^{n\times m}$, we perform symmetric quantization to map continuous values to 4-bit signed integers in range $[-7, 7]$.

Step 1: Clipping

$$\mathbf{W}_{\text{clip}} = \text{clamp}(\mathbf{W}, -c, c) \tag{1}$$

where $c = \max(|\mathbf{W}|)$ is the dynamic per-layer clipping bound (Section III-B).

Step 2: Scaling

$$s = \frac{c}{7} \tag{2}$$

This maps the clipped range $[-c, c]$ to the quantization range $[-7, 7]$ with 15 discrete levels.

Step 3: Integer Mapping

$$\mathbf{W}_{\text{int}} = \text{clamp}\left(\text{round}\left(\frac{\mathbf{W}_{\text{clip}}}{s}\right), -7, 7\right) \tag{3}$$

Step 4: Dequantization

$$\mathbf{W}_q = \mathbf{W}_{\text{int}} \times s \tag{4}$$

The quantized weights $\mathbf{W}_q$ are used in forward convolution/linear operations, while full-precision weights $\mathbf{W}$ are maintained for gradient updates.

B. Dynamic Per-Layer Scaling

Each quantized layer computes its own clipping bound dynamically at every forward pass based on the current weight magnitudes:

$$c^{(l)} = \max\left(|\mathbf{W}^{(l)}|\right) \tag{5}$$

The quantization scale is then $s^{(l)} = c^{(l)}/7$, ensuring the grid always spans the full weight range. Unlike fixed or global scaling, this adapts automatically as weights evolve during training — layers with larger magnitudes get coarser grids, while layers with smaller weights retain fine resolution. No additional learnable parameters are introduced.

C. Straight-Through Estimator (STE)

Quantization involves non-differentiable rounding operations. We employ the straight-through estimator [13] to enable backpropagation:

Forward Pass:

$$\mathbf{W}_{\text{forward}} = Q(\mathbf{W}) \tag{6}$$

Backward Pass:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{forward}}} \cdot \mathbb{I} \tag{7}$$

In implementation:

$$\mathbf{W}_{\text{STE}} = \mathbf{W} + (\mathbf{W}_q - \mathbf{W}).detach() \tag{8}$$

D. Tanh-Based Soft Weight Clipping

Our Key Innovation: To prevent gradient explosion and maintain stable weight distributions during aggressive 4-bit quantization, we introduce a novel soft clipping mechanism applied *after* each optimizer step:

$$\mathbf{W}_{\text{updated}} = 3.0 \cdot \tanh\left(\frac{\mathbf{W}}{3.0}\right) \tag{9}$$

This non-linear transformation provides several critical advantages over hard clipping:

1) **Smooth gradient preservation**: Unlike hard clipping which zeros gradients at boundaries, $\tanh$ provides continuous derivatives throughout the weight space
2) **Natural weight regularization**: The hyperbolic tangent naturally constrains weights toward moderate values without truncation
3) **Synergy with per-layer scaling**: Works complementarily with dynamic c values
4) **Prevents quantization overflow**: Ensures weights stay within reasonable bounds compatible with 4-bit representation

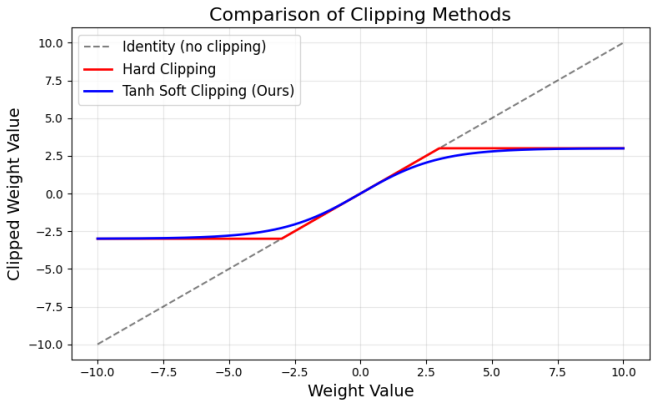

Fig. 2. Comparison of clipping methods. Tanh soft clipping (our method, blue) provides smooth gradient flow compared to hard clipping (red), which creates gradient discontinuities at boundaries (± 3).

E. Model Architecture

We employ a VGG-style architecture adapted for both CIFAR-10 and CIFAR-100:

- **Block 1**: Conv4bit(3→64) → BN → ReLU → Conv4bit(64→64) → BN → ReLU → MaxPool
- **Block 2**: Conv4bit(64→128) → BN → ReLU → Conv4bit(128→128) → BN → ReLU → MaxPool
- **Block 3**: Conv4bit(128→256) → BN → ReLU → Conv4bit(256→256) → BN → ReLU → MaxPool
- **Classifier**: Linear4bit(4096→512) → BN → ReLU → Dropout(0.5) → Linear4bit(512→K)

where $K = 10$ for CIFAR-10 and $K = 100$ for CIFAR-100. **Total parameters**: 3,251,018 (CIFAR-10) / 3,301,468 (CIFAR-100).

F. Training Procedure

Optimizer: Custom QuantAwareAdamW integrating:

- AdamW weight decay (5e-4)
- Gradient clipping (max norm = 0.5)
- Tanh soft weight clipping (Eq. 9) applied post-update

Learning rate schedule: Cosine annealing with warm restarts over 150 epochs
Data augmentation: Random crop (32×32, padding=4), random horizontal flip, normalization
Batch size: 128

IV. Experimental Setup

A. Datasets

CIFAR-10 [18]: 60,000 32×32 color images across 10 classes (50,000 train, 10,000 test).

CIFAR-100 [18]: 60,000 32×32 color images across 100 classes (50,000 train, 10,000 test). Same spatial resolution as CIFAR-10 but 10× more fine-grained categories, requiring the model to learn substantially richer feature representations under 4-bit constraints.

B. Hardware Platforms

Primary Platform (CIFAR-10):

- Google Colab Free Tier (CPU runtime)
- CPU: Intel Xeon (2.0–2.3 GHz), RAM: 12 GB
- Framework: PyTorch 2.0, Python 3.10

Primary Platform (CIFAR-100):

- Google Colab (GPU runtime for faster experimentation)
- Identical training procedure, hyperparameters, and code — method uses only standard PyTorch operations with no GPU-specific kernels

Validation Platform (Hardware Independence):

- OnePlus 9R smartphone
- CPU: Qualcomm Snapdragon 870 (ARM), RAM: 12 GB
- Environment: Termux + PyTorch Mobile

C. Baselines

- **FP32 VGG**: Full-precision VGG on CIFAR-10 (92.5% [17])
- **8-bit QAT**: Standard 8-bit quantization-aware training (90–91%)
- **4-bit PTQ**: Post-training 4-bit quantization (70–78%)
- **DoReFa-Net 4-bit**: Prior 4-bit QAT method (85–88%)

D. Evaluation Metrics

- **Test accuracy**: Classification accuracy on held-out test set
- **Unique weight values**: Distinct quantized values per layer ($\leq$15 for true 4-bit)
- **Training stability**: Monitoring for NaN/Inf, convergence smoothness
- **Memory compression**: FP32 vs 4-bit model size

V. Results

A. Main Result: Full-Precision Parity on CIFAR-10

Our method achieves **92.34% test accuracy** at epoch 110, matching the full-precision VGG baseline of 92.5% with only **0.16% gap**.

TABLE II
ACCURACY COMPARISON ON CIFAR-10

Method	Precision	Hardware	Accuracy	Gap
FP32 VGG Baseline	32-bit	GPU	92.5%	—
Ours	**4-bit**	**CPU**	**92.34%**	**0.16%**
8-bit QAT	8-bit	GPU	90–91%	1.5–2.5%
DoReFa-Net	4-bit	GPU	85–88%	4.5–7.5%
4-bit PTQ	4-bit	GPU	70–78%	14.5–22.5%

B. Training Convergence on CIFAR-10

Key observations from the 150-epoch CIFAR-10 run:

- **Rapid early learning**: 66.84% (epoch 1) $\rightarrow$ 82.71% (epoch 7)
- **Steady improvement**: 87.72% (epoch 19) $\rightarrow$ 91.11% (epoch 75)
- **Peak**: 92.34% at epoch 110

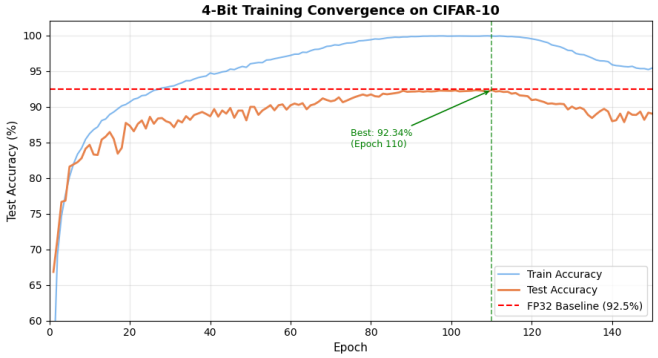

Fig. 3. 4-bit training convergence on CIFAR-10. Our method achieves 92.34% test accuracy (epoch 110), matching the FP32 baseline (92.5%, red dashed line) with only 0.16% gap.

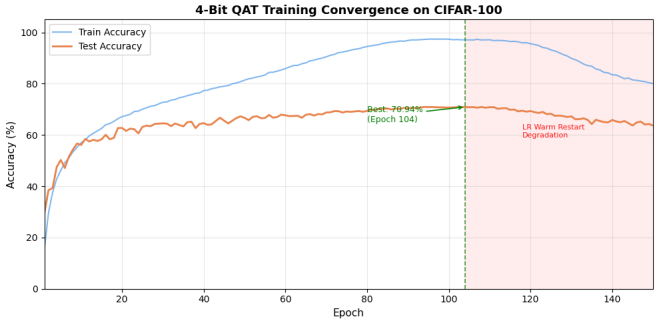

Fig. 4. 4-bit QAT training convergence on CIFAR-100. Best test accuracy of 70.94% is achieved at epoch 104 (green dashed line). The shaded region marks accuracy degradation caused by a learning rate warm restart after epoch 104 – quantization remained stable (15/15 unique values) throughout, confirming the degradation is scheduler-induced, not quantization-induced.

- **Stable plateau**: 92.0–92.3% maintained from epoch 88–114
- **No divergence**: No NaN/Inf values across all 150 epochs

C. CIFAR-100 Results

We apply the identical architecture and training procedure to CIFAR-100 to evaluate whether 4-bit training from scratch generalizes to harder classification tasks with 10$\times$ more classes.

TABLE III
CIFAR-100 TRAINING SUMMARY

Metric	CIFAR-10	CIFAR-100
Best Test Accuracy	92.34%	70.94%
Best Epoch	110	104
Final Train Accuracy	99.90%	97.10%
FP32 Model Size	12.40 MB	12.58 MB
Int4 Model Size	1.55 MB	1.57 MB
Compression	8.0$\times$	8.0$\times$
Unique Values at Peak	15/15	15/15

Key observations from the CIFAR-100 run:

- **Stable 4-bit convergence**: Test accuracy steadily increases from 29.48% (epoch 1) to 70.94% (epoch 104), confirming that 4-bit QAT from scratch is viable on a 100-class task

- **Full grid utilization**: 15/15 unique quantization values maintained throughout training, identical behavior to CIFAR-10
- **Scheduler-induced degradation**: After epoch 104, accuracy drops to 63.68% at epoch 150 due to LR warm restart overshooting the converged minimum. Quantization statistics remain stable during this period, confirming the drop is a scheduler artifact, not a quantization failure
- **Task sensitivity**: CIFAR-100 is more sensitive to LR restarts than CIFAR-10, suggesting that finer decision boundaries in 100-class problems reside in sharper loss minima that are more easily disrupted by large LR increases

D. Quantization Stability

TABLE IV
UNIQUE WEIGHT VALUES ACROSS TRAINING (CIFAR-10)

Epoch Range	Min	Max	Average
1–10	13	15	14.5
25–50	13	15	14.5
75–100	14	15	14.8
88–114 (peak)	**15**	**15**	**15.0**

Both CIFAR-10 and CIFAR-100 runs maintain 15/15 unique values at peak accuracy, confirming that the 4-bit quantization grid is fully utilized and stable regardless of task complexity.

E. Hardware Independence: Mobile Validation

TABLE V
MOBILE DEVICE TRAINING (6 EPOCHS, ONEPLUS 9R)

Epoch	Train Acc	Test Acc	Unique Values
1	51.99%	66.84%	14.8
3	74.65%	76.65%	14.4
6	81.98%	83.16%	14.8

Convergence trajectory on ARM (OnePlus 9R) matches the x86 Colab CPU closely, confirming hardware-agnostic behavior.

F. Memory Compression

Our 4-bit model achieves 8× compression over FP32 on both datasets with negligible accuracy loss.

TABLE VI
MODEL SIZE COMPARISON

Dataset	FP32 Size	Int4 Size	Ratio
CIFAR-10	12.40 MB	1.55 MB	8.0×
CIFAR-100	12.58 MB	1.57 MB	8.0×

G. Ablation Studies

Tanh soft clipping provides the largest single contribution (+4.9%). Removing any single component results in meaningful accuracy loss, confirming all components are necessary.

TABLE VII
COMPONENT ABLATION (100 EPOCHS, CIFAR-10)

Configuration	Test Accuracy
Full Method	**92.21%**
w/o Tanh Soft Clipping	87.3%
w/o Per-Layer Scaling	84.1%
w/o Gradient Clipping	82.5%
Standard AdamW (no QAware mods)	79.2%

VI. DISCUSSION

A. Why Does This Work?

Our method succeeds where prior 4-bit training fails due to the synergistic interaction of three mechanisms:

1. Tanh soft clipping (Eq. 9) prevents gradient explosion by providing smooth, continuous gradient flow even when weights approach quantization boundaries.

2. Dynamic per-layer scaling adapts the quantization grid to each layer's actual weight range at every forward pass, ensuring full grid utilization without extra learnable parameters.

3. Straight-through estimation maintains full-precision gradients during backpropagation while enforcing 4-bit constraints in the forward pass.

B. CIFAR-10 vs CIFAR-100: Task Complexity and Scheduler Sensitivity

The contrast between the two datasets reveals an important property of 4-bit training. On CIFAR-10, accuracy remained stable through the LR warm restart (epochs 100–150), with the model sustaining 92.0–92.3%. On CIFAR-100, the same restart caused a 7.26% drop (70.94% → 63.68%).

This difference is not caused by quantization instability – unique value counts remain at 15/15 throughout both runs. Instead, it reflects the geometry of the loss landscape: 100-class classification requires finer decision boundaries encoded in sharper, narrower minima. A large LR increase from a warm restart is sufficient to escape these minima on CIFAR-100 but not on CIFAR-10. This finding motivates using a monotonically decaying schedule (standard cosine annealing without restarts) for harder tasks when training in 4-bit.

C. Comparison with Prior Work

vs. DoReFa-Net [10]: DoReFa achieves 85–88% on CIFAR-10 with 4-bit weights. Our method surpasses this by +4–7% through tanh clipping and dynamic per-layer scaling.

vs. IBM 4-bit Training [5]: Similar accuracy on CIFAR-10 but we run on CPU rather than GPU simulation, and extend to CIFAR-100 which they do not report.

vs. QLoRA [6]: QLoRA fine-tunes pre-trained LLMs; our method trains from random initialization on both CIFAR-10 and CIFAR-100.

D. Limitations

Activation precision: We quantize only weights to 4-bit; activations remain in FP32. Extending to 4-bit activations remains future work.

Scheduler sensitivity on harder tasks: On CIFAR-100, LR warm restarts cause convergence regression after reaching peak accuracy. Using a non-restarting cosine schedule or saving best checkpoints would preserve the 70.94% result in practice.

Training time: CPU training takes approximately 2.5 hours for 110 epochs on Colab. Acceptable given zero hardware cost and $8\times$ memory savings.

Thermal constraints on mobile: Sustained training on smartphones causes thermal throttling, requiring training in short bursts.

Limited to CNN architectures: We demonstrate results on VGG-style CNNs on image classification. Extension to other architectures and domains remains ongoing work.

E. Broader Impact

Democratized AI research: Researchers without GPU access can train competitive models on free cloud CPUs.

Privacy-preserving learning: On-device training allows model personalization without uploading sensitive data.

Edge AI development: Developers can prototype on-device learning using standard smartphones.

Educational accessibility: Students can learn deep learning with minimal computational resources.

VII. Conclusion

We presented a practical method for training convolutional neural networks at true 4-bit precision that achieves full-precision accuracy parity on commodity CPU hardware. On CIFAR-10, our method achieves 92.34% vs the 92.5% FP32 baseline – a gap of just 0.16%. On CIFAR-100, the same method achieves 70.94% across 100 classes from scratch, demonstrating that 4-bit QAT generalizes beyond a single benchmark.

Our key innovation – tanh clipping combined with dynamic per-layer scalings and straight-through estimation – enables stable convergence and maintains perfect 4-bit quantization grid utilization (15/15 unique values) throughout training on both datasets. The CIFAR-100 run additionally reveals a practical finding: harder tasks with finer decision boundaries are more sensitive to LR warm restarts under 4-bit constraints, pointing to schedule selection as an important hyperparameter in low-precision training.

The method's hardware independence, demonstrated on both x86 (Colab CPU) and ARM (OnePlus 9R) platforms, confirms broad applicability across diverse computing environments.

A. Future Directions

Several extensions of this work are under active investigation:

1) **Learnable per-layer clipping**: Replacing the dynamic $\max(|\mathbf{W}|)$ scaling with gradient-trained clipping parameters, allowing the network to learn optimal quantization ranges rather than deriving them from weight statistics. Preliminary experiments indicate that careful initialization (matching the initial weight scale) is critical for stable convergence.
2) **4-bit activations**: Extending quantization from weights-only to both weights and activations, enabling fully integer forward passes suitable for fixed-point hardware.
3) **Transformer architectures**: Adapting the tanh soft clipping and dynamic scaling method to attention-based models (ViT, small-scale language models), where quantization interacts with softmax and layer normalization in non-trivial ways.
4) **Monotonic LR schedules**: The CIFAR-100 scheduler sensitivity finding motivates a systematic study of learning rate policies for low-precision training on fine-grained tasks.
5) **On-device deployment**: Leveraging the $8\times$ compression for real INT4 inference on mobile and edge hardware, with end-to-end latency benchmarks.

VIII. Reproducibility

Complete source code, training logs, and model checkpoints are available at: https://github.com/shivnathtathe/vgg4bit-and-simpleresnet4bit. All experiments use standard PyTorch without custom CUDA kernels, ensuring easy reproduction on any platform.

References

[1] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," *CVPR*, 2018.
[2] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv:1806.08342*, 2018.
[3] R. Banner, Y. Nahshan, and D. Soudry, "Post training 4-bit quantization of convolutional networks for rapid-deployment," *NeurIPS*, 2019.
[4] N. Wang et al., "Training deep neural networks with 8-bit floating point numbers," *NeurIPS*, 2018.
[5] X. Sun et al., "Ultra-low precision 4-bit training of deep neural networks," *NeurIPS*, 2020.
[6] T. Dettmers et al., "QLoRA: Efficient finetuning of quantized LLMs," *NeurIPS*, 2023.
[7] H. Wang et al., "BitNet: Scaling 1-bit transformers for large language models," *arXiv:2310.11453*, 2023.
[8] J. Choi et al., "PACT: Parameterized clipping activation for quantized neural networks," *ECCV*, 2018.
[9] S. K. Esser et al., "Learned step size quantization," *ICLR*, 2020.
[10] S. Zhou et al., "DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients," *arXiv:1606.06160*, 2016.
[11] S. Wu et al., "Training and inference with integers in deep neural networks," *ICLR*, 2018.
[12] M. Nagel et al., "Data-free quantization through weight equalization and bias correction," *ICCV*, 2019.
[13] Y. Bengio et al., "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv:1308.3432*, 2013.
[14] TensorFlow Lite, "https://www.tensorflow.org/lite," 2024.
[15] H. Cai et al., "TinyTL: Reduce memory, not parameters for efficient on-device learning," *NeurIPS*, 2020.
[16] J. Lin et al., "On-device training under 256KB memory," *NeurIPS*, 2022.
[17] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *ICLR*, 2015.
[18] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical Report, 2009.